DOSSIER TECHNIQUE — PROJET ACTURISK

Projet : Plateforme d'Assurance Santé
Classe / Groupe : 4Actuariat - Groupe 7
Membres : Racha Khammassi, Cyrine Boukhit, Mariem Cherif, Zeyneb Chouchene
Lien du dépôt Git : https://github.com/cyrine-bk/ActuRisk-4Actuariat-g7.git

1. Description du sujet et Problématique

La plateforme **ActuRisk** est une solution d'assurance santé qui connaît une croissance massive de son volume de simulations et de sinistres.

L'architecture monolithique d'origine montre ses limites :

- **Couplage fort :** Toutes les entités (Assurés, Devis, Contrats, Sinistres) partagent la même base de données, bloquant les évolutions.
- **Scalabilité inefficace :** Les pics de calcul actuariel obligent à dupliquer toute l'application.
- **Blocage synchrone :** Les calculs lourds de prédiction (ML) bloquent la déclaration de sinistre pour l'utilisateur.
- **Goulot d'équipe :** Conflits Git fréquents et impossibilité d'intégrer des technologies variées.

Solution : Migration vers une architecture microservices basée sur le **Domain-Driven Design (DDD)**, découpant le système en contextes métier autonomes et découplés, sécurisés par **Keycloak** et articulés autour de 5 microservices métiers.

2. Acteurs et Fonctionnalités (Cas d'utilisation)

L'application est conçue pour trois profils d'utilisateurs authentifiés via Keycloak :

- **Assuré / Client :** Peut effectuer une simulation de tarification, souscrire un contrat d'assurance santé, payer ses cotisations et déclarer un sinistre.
- **Gestionnaire Sinistres :** Instruit les dossiers, valide et transmet les indemnisations pour versement.
- **Actuaire / Analyste :** Ajuste les grilles tarifaires, paramètre les facteurs de risque et ré-entraîne les modèles prédictifs pour l'intelligence actuarielle.

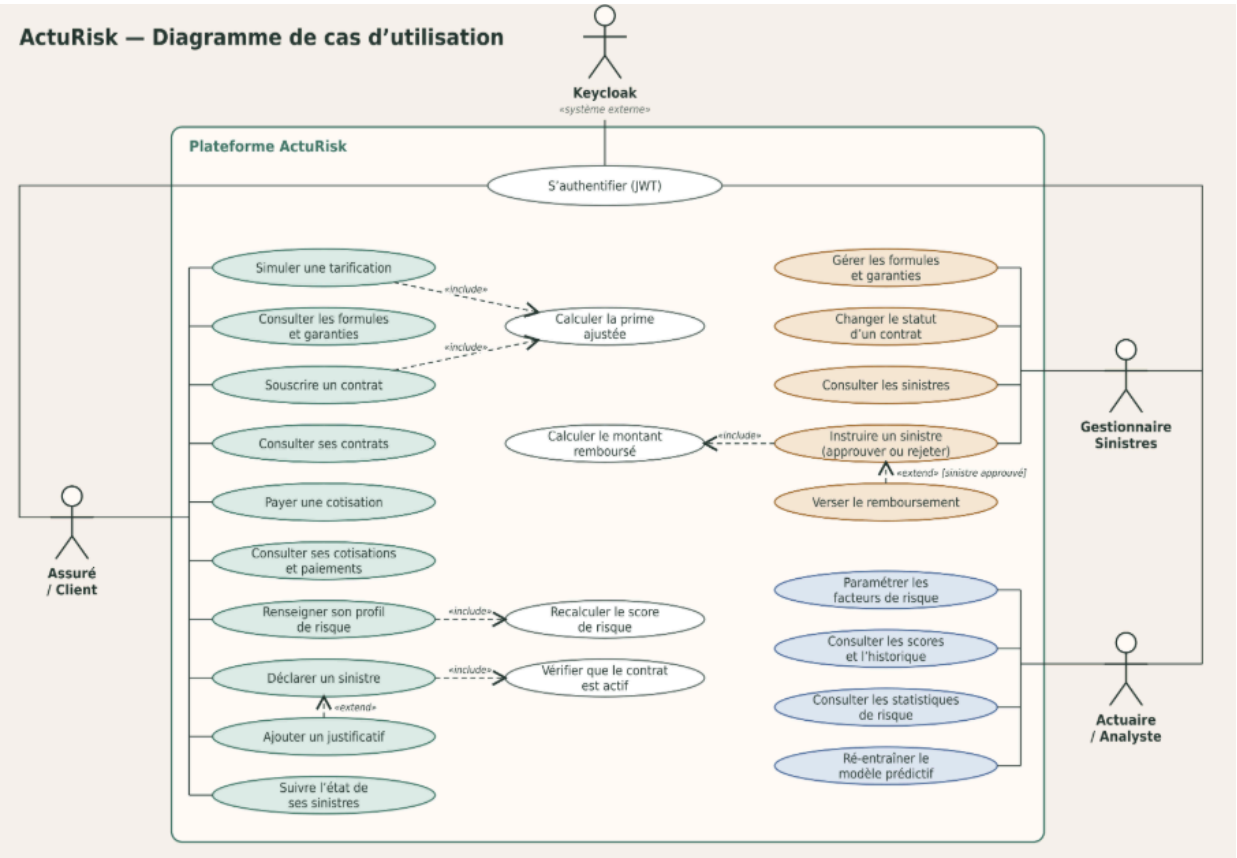


Figure 1 – Diagramme de cas d'utilisation général

3. Découpage en Microservices (Bounded Contexts)

Le système est découpé en 5 microservices autonomes :

1. **`billing-service` (MS1 - Port 8081) :** Gestion financière (cotisations, paiements, versements de remboursements et détection des impayés).
2. **`policy-service` (MS2 - Port 8082) :** Gestion des formules, garanties et contrats d'assurance.
3. **`claim-service` (MS3 - Port 8083) :** Gestion du cycle de vie des déclarations de sinistres et justificatifs.
4. **`risk-service` (MS4 - Port 8084) :** Gestion des profils de risque des assurés et paramétrage actuariel.
5. **`predictive-scoring-service` (MS5 - Port 8085) :** Moteur prédictif (FastAPI / Python) pour le calcul des scores et de la prime ajustée.

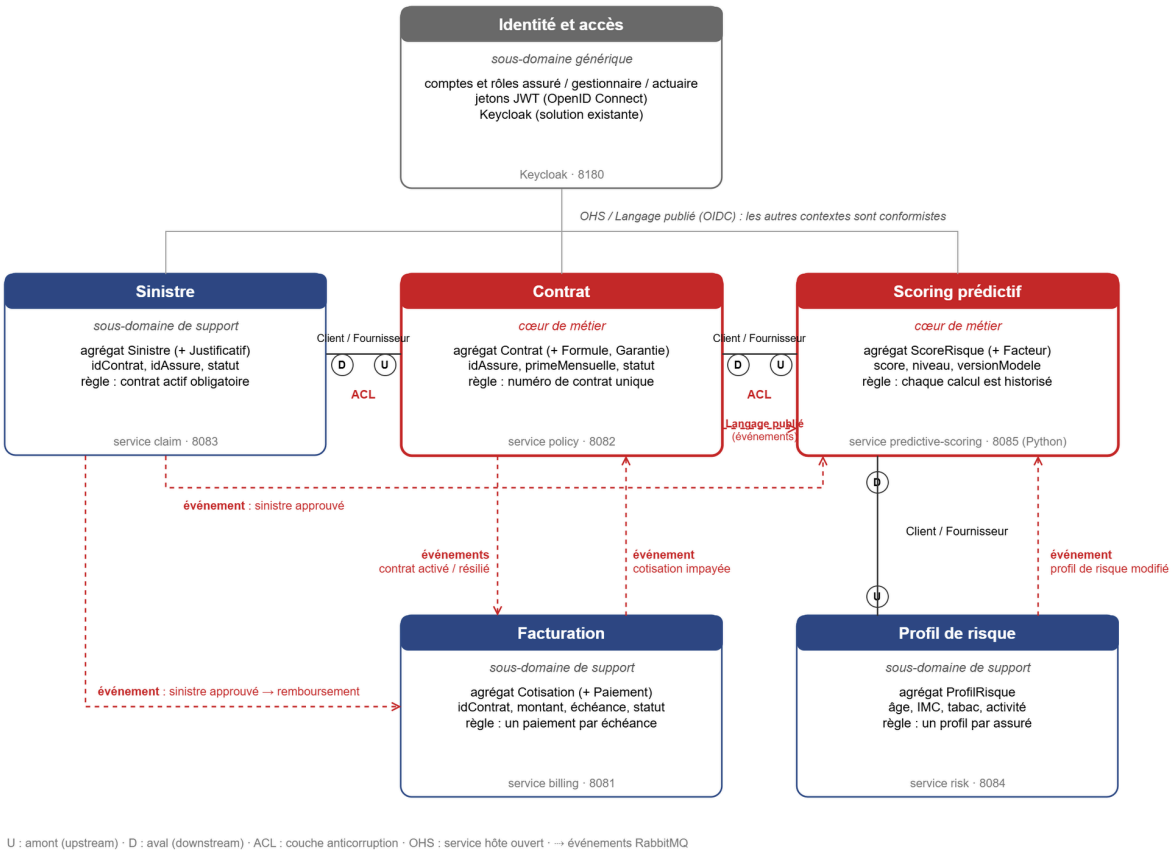


Figure 2 – Carte de Contexte

4. Données gérées par chaque service

Le choix s'est porté sur une architecture de persistance adaptée aux besoins de chaque service :

- **MySQL :** Utilisé pour le `billing-service` (MS1). Garantie transactionnelle stricte indispensable pour l'historique financier et les paiements.
- **PostgreSQL (SQL) :** Utilisé pour le `policy-service` (MS2), le `claim-service` (MS3) et le `risk-service` (MS4). Garantit une intégrité transactionnelle (ACID) stricte indispensable pour les contrats, les sinistres/remboursements et les profils de risque.
- **MongoDB (NoSQL) :** Utilisé par le `predictive-scoring-service` (MS5). Permet de stocker et manipuler facilement les matrices de données dynamiques et les résultats du modèle prédictif (FastAPI / Python).

5. Interactions entre les services

Les microservices communiquent de manière découplée :

- **Communication Synchrone (REST / OpenFeign) :**
  - Le `claim-service` interroge le `policy-service` pour vérifier la validité et le taux de couverture du contrat avant de traiter un remboursement.
  - Le `policy-service` interroge le `predictive-scoring-service` pour obtenir la prime ajustée selon le score.
  - Le `predictive-scoring-service` interroge le `risk-service` pour récupérer le profil de risque.
- **Communication Asynchrone (RabbitMQ / Event-Driven) :**
  - Le `policy-service` publie les événements `contrat.active` et `contrat.resilie` consommés par le `billing-service` (pour l'échéancier) et le `predictive-scoring-service`.
  - Le `claim-service` publie l'événement `sinistre.approuve` consommé par le `billing-service` (pour déclencher le remboursement) et le `predictive-scoring-service` (pour mettre à jour l'historique).
  - Le `billing-service` publie `cotisation.impayee` pour permettre au `policy-service` de suspendre le contrat.

6. Diagramme d'Architecture Globale

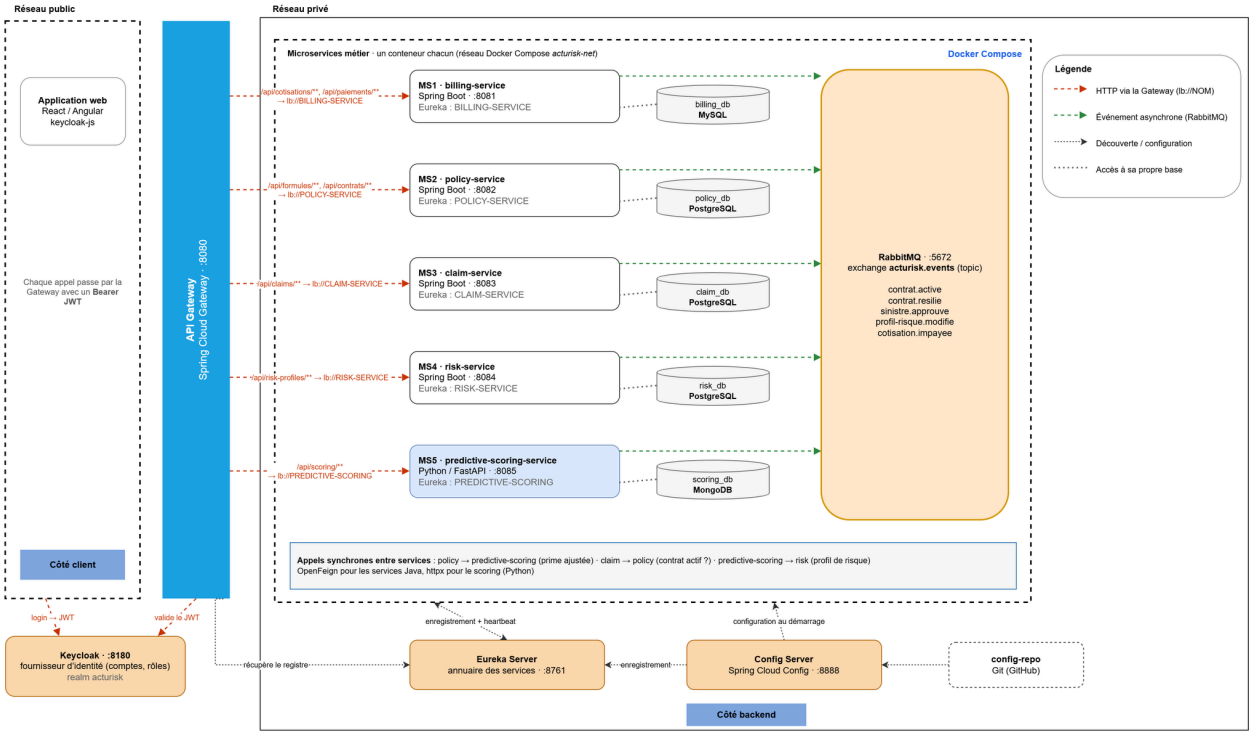


Figure 3 – Diagramme d'architecture globale

Le Front-end (React) passe par l'API Gateway (Spring Cloud Gateway) sécurisée par **Keycloak** (gestion des comptes, rôles et tokens JWT sur le port 8180). La Gateway s'appuie sur **Eureka** pour router les requêtes vers les 5 microservices. Chaque microservice possède sa propre base de données isolée. Le tout est orchestré via **Docker Compose**.

7. Structure du Dépôt Git :

```
None
ActuRisk-4Actuariat-g7/
|-- Backend/
|   |-- billing-service/            Service Facturation (MS1)
|   |-- policy-service/             Service Polices (MS2)
|   |-- claim-service/              Service Sinistres (MS3)
|   |-- risk-service/               Service Risque (MS4)
|   |-- predictive-scoring/         Service Commun / ML (MS5)
|-- Frontend/                       Application cliente Web
|-- Documentation/
|   |-- Presentation/
|   |-- Diagrams/
```

8. Microservices

A. Service Facturation (`billing-service` - MS1)

Le microservice billing-service gère les cotisations mensuelles des contrats, leur paiement par les assurés, le versement des remboursements de sinistres et la détection des impayés. Il est développé avec Spring Boot (port 8081). Il utilise une base MySQL (billing_db).
Les paiements sont des données financières : MySQL garantit les transactions et l'intégrité des montants, et conserve l'historique des paiements de façon durable. Cette base n'est accessible que par ce microservice.

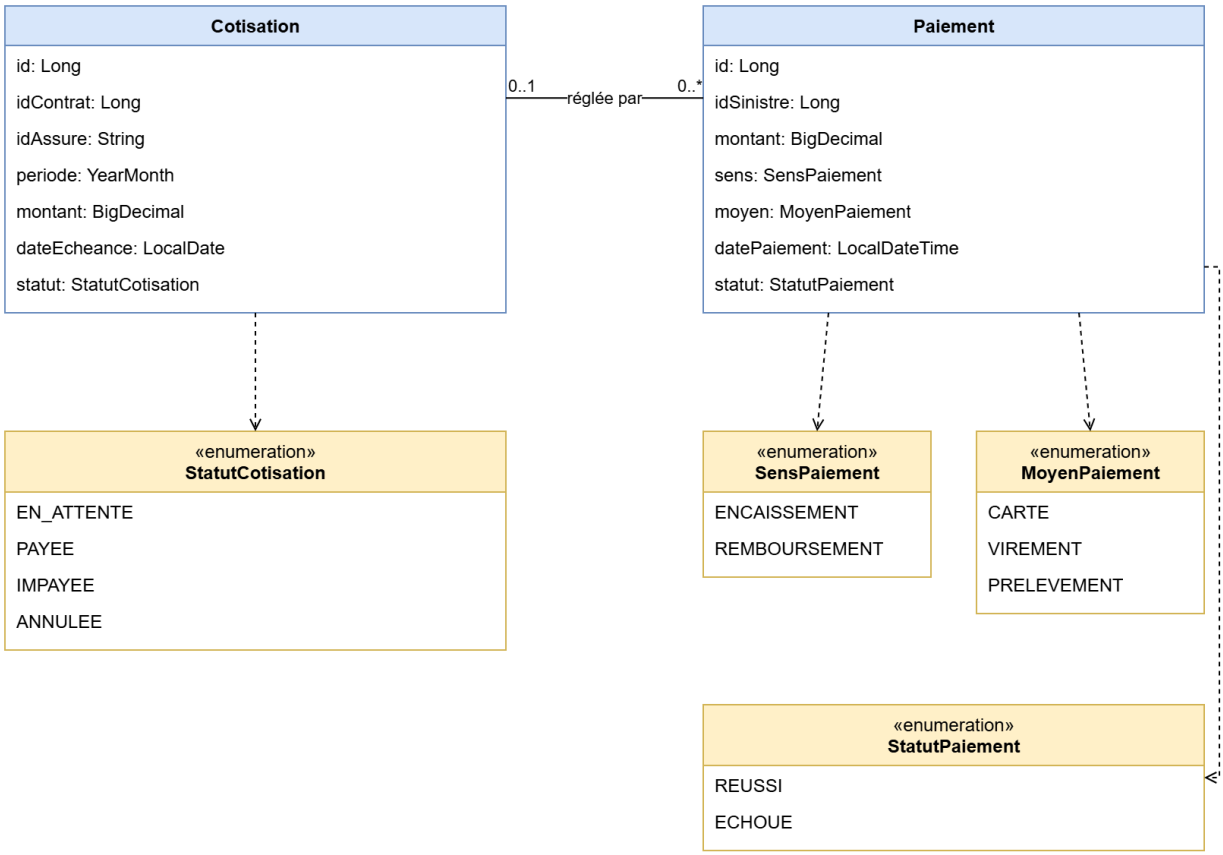


Figure 4 – Diagramme de classes du microservice billing-service

Les attributs idContrat et idSinistre référencent des données de policy-service et de claim-service par leur identifiant. L'identifiant idAssure est celui de l'utilisateur dans Keycloak : il est lu dans le jeton JWT, déjà vérifié par la Gateway. Aucun microservice ne gère les comptes. Chaque événement reçu est identifié par un idEvenement : un message reçu deux fois n'est traité qu'une seule fois.

Contrat d'API :

Méthode	Endpoint	Description	Codes
GET	/api/cotisations?idContrat=&idAssure=&statut=	Lister les cotisations	200
GET	/api/cotisations/{id}	Détail d'une cotisation	200, 404
POST	/api/cotisations/{id}/paiement	Payer une cotisation	201, 402, 404, 409
GET	/api/paiements?idAssure=&sens=	Lister les paiements	200
GET	/api/paiements/{id}	Détail d'un paiement	200, 404

402 : paiement refusé ; 409 : cotisation déjà payée ou annulée.

Evénements:

Événement	Sens	Contenu	Effet
ContratActive (contrat.active)	Reçu de policy-service	idContrat, idAssure, primeMensuelle, dateDebut	Création de l'échéancier
ContratResilie (contrat.resilie)	Reçu de policy-service	idContrat, dateFin	Annulation des cotisations futures
SinistreApprouve (sinistre.approuve)	Reçu de claim-service	idSinistre, idAssure, montantRembourse	Paiement REMBOURSEMENT
CotisationImpayee (cotisation.impayee)	Publié	idContrat, idCotisation, nombreImpayes	policy-service suspend le contrat

Interactions:

Service	Mode	Objet	En cas d'indisponibilité
API Gateway (client)	Synchrone (HTTP)	Paiement d'une cotisation	—
policy-service	Asynchrone (RabbitMQ)	ContratActive, ContratResilie / CotisationImpayee	Message conservé dans la file
claim-service	Asynchrone (RabbitMQ)	SinistreApprouve	Message conservé dans la file

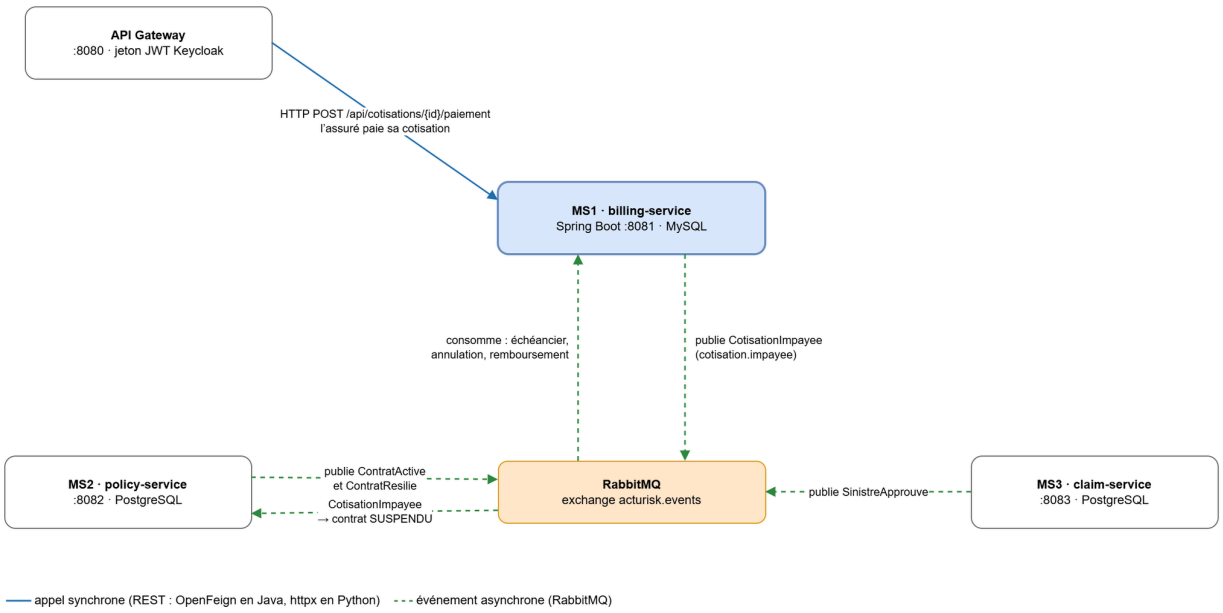


Figure 5 – Interactions du microservice billing-service

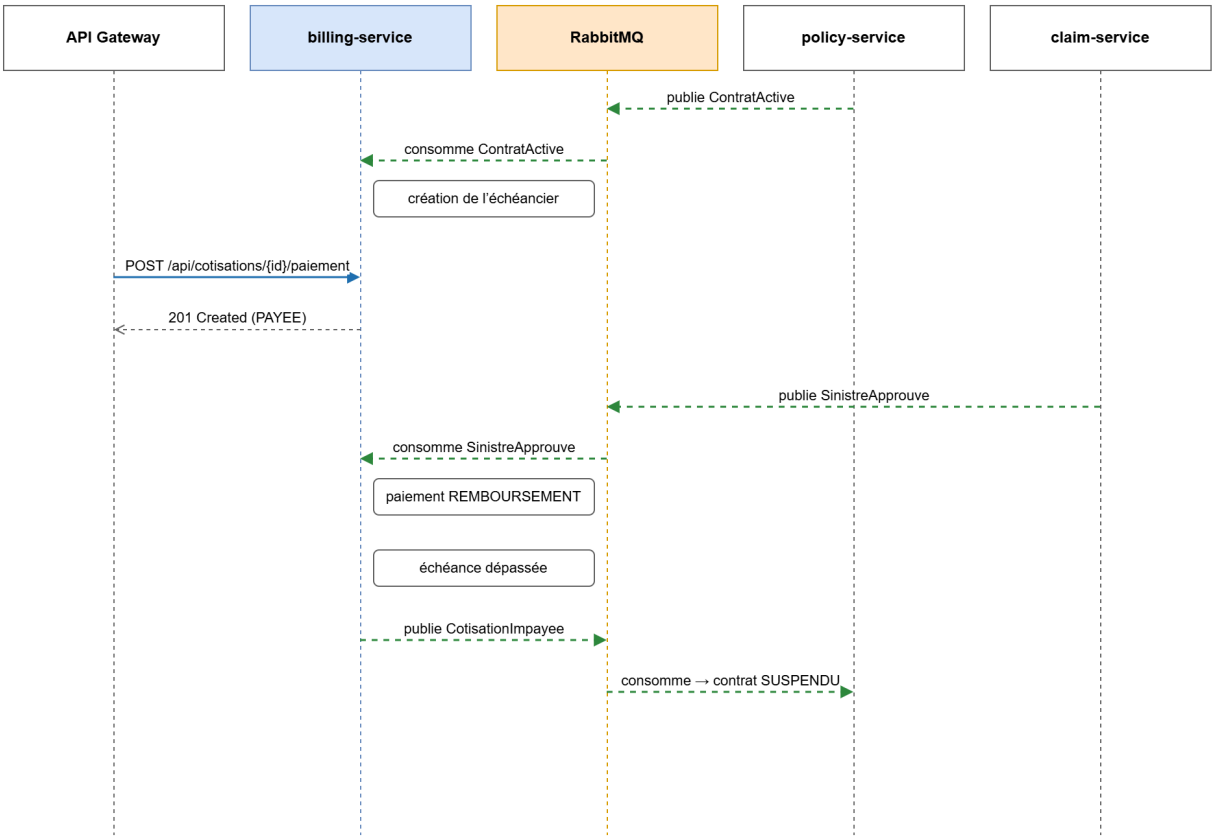


Figure 6 – Diagramme de séquence des cotisations et du remboursement

B. Service Polices (`policy-service` - MS2)

Le microservice policy-service gère les formules d'assurance, leurs garanties et les contrats souscrits par les assurés. Il est développé avec Spring Boot (port 8082).
Il utilise une base PostgreSQL (policy_db). Les données sont relationnelles (une formule comprend des garanties, un contrat est souscrit selon une formule) et soumises à des contraintes fortes : numéro de contrat unique, statuts et dates. PostgreSQL garantit l'intégrité des données et les transactions, et conserve les contrats de façon durable. Cette base n'est accessible que par ce microservice

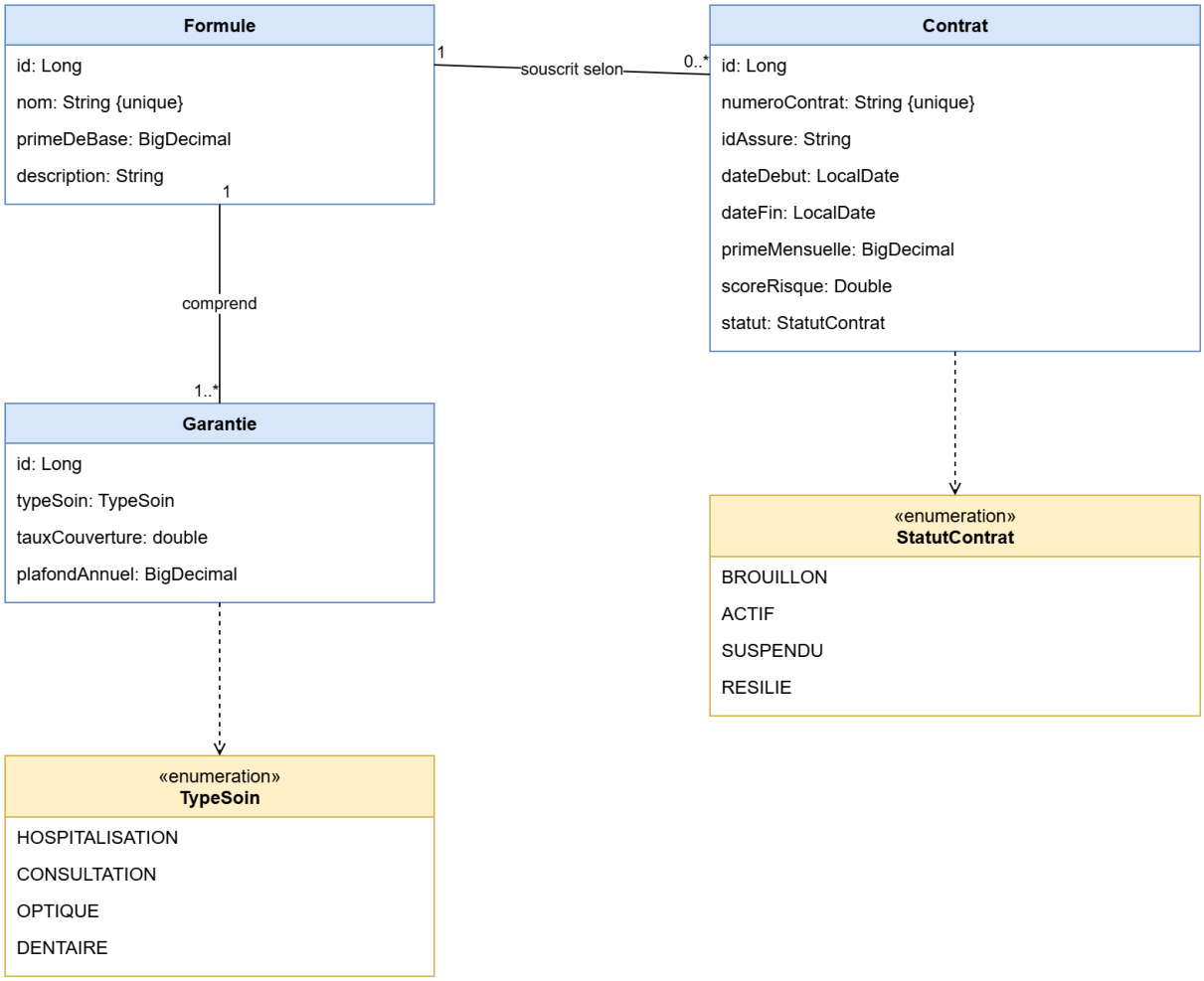


Figure 7 – Diagramme de classes du microservice policy-service

L'identifiant idAssure est celui de l'utilisateur dans Keycloak : il est lu dans le jeton JWT, déjà vérifié par la Gateway. Aucun microservice ne gère les comptes. La formule porte le prix catalogue (primeDeBase) ; le contrat garde le prix réellement payé (primeMensuelle), ajusté selon le score de risque.

Contrat d'API:

Méthode	Endpoint	Description	Codes
GET	/api/formules	Lister les formules	200
GET	/api/formules/{id}	Détail d'une formule	200, 404
POST	/api/formules	Créer une formule	201, 400, 409
PUT	/api/formules/{id}	Modifier une formule	200, 400, 404
GET	/api/contrats?idAssure=&statut=	Lister les contrats	200
GET	/api/contrats/{id}	Détail d'un contrat	200, 404
POST	/api/contrats	Souscrire un contrat	201, 400, 404
PATCH	/api/contrats/{id}/statut	Changer le statut	200, 404, 409

404 : formule ou contrat introuvable ; 409 : nom de formule déjà utilisé ou changement de statut interdit.

Événements :

Événement	Sens	Contenu	Effet
ContratActive (contrat.active)	Publié	idContrat, idAssure, primeMensuelle, dateDebut	billing crée l'échéancier ; predictive-scoring met à jour le portefeuille
ContratResilie (contrat.resilie)	Publié	idContrat, dateFin	billing annule les cotisations futures
CotisationImpayee (cotisation.impayee)	Reçu de billing-service	idContrat, nombreImpayes	Contrat suspendu

Interactions :

Service	Mode	Objet	En cas d'indisponibilité
predictive-scoring-service (appelé)	Synchrone (OpenFeign)	Obtenir la prime ajustée	Prime de base de la formule (Resilience4j)
claim-service (appelant)	Synchrone (OpenFeign)	Statut et taux de couverture d'un contrat	—
billing-service	Asynchrone (RabbitMQ)	ContratActive, ContratResilie / CotisationImpayee	Message conservé dans la file
predictive-scoring-service	Asynchrone (RabbitMQ)	ContratActive	Message conservé dans la file

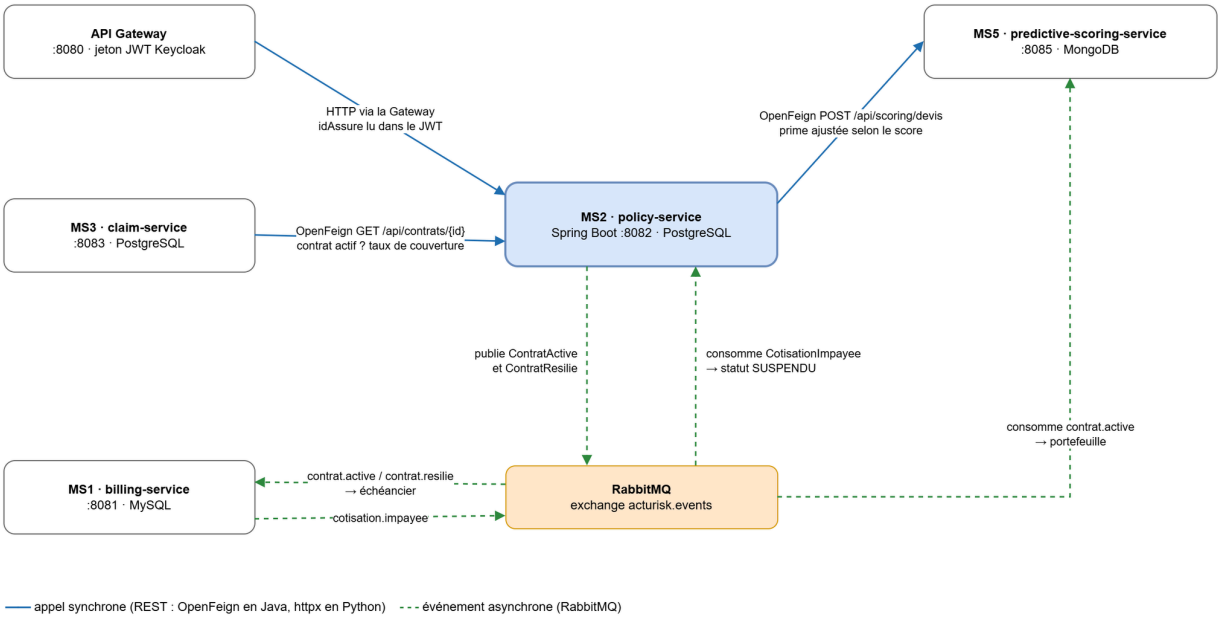


Figure 8 – Interactions du microservice policy-service

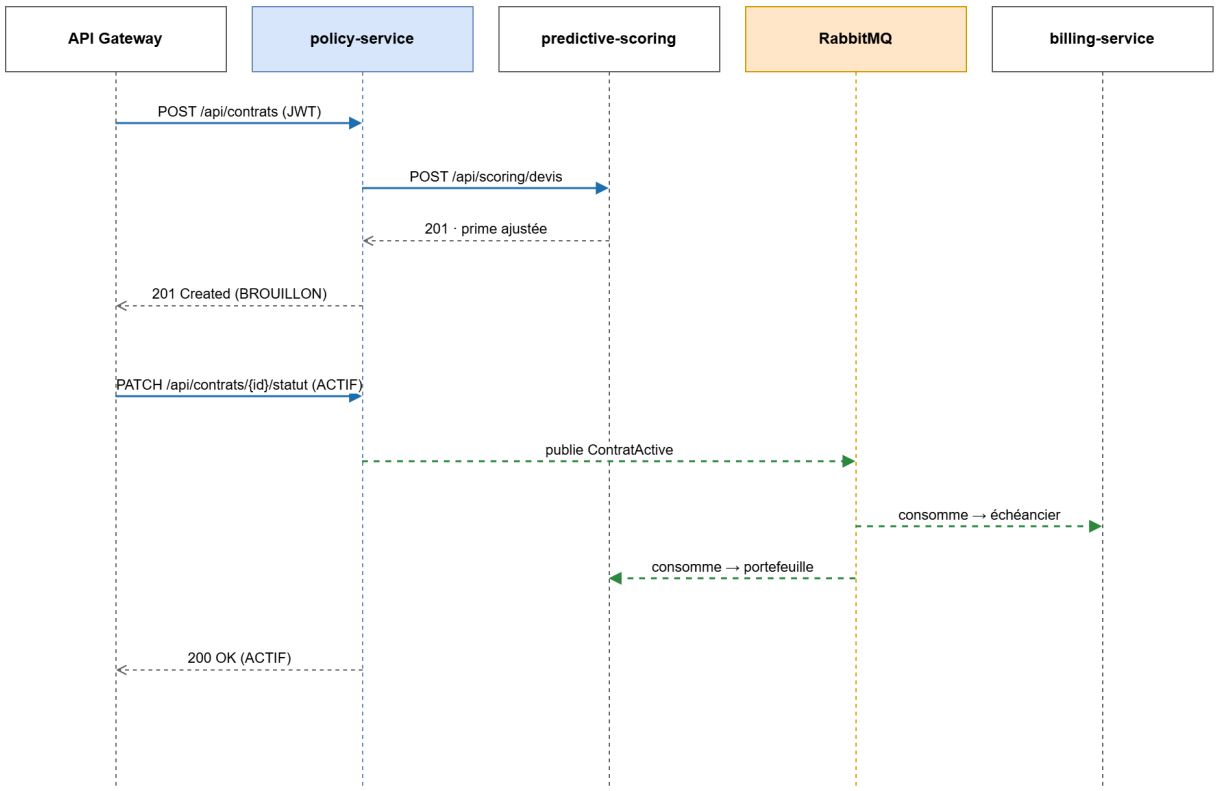


Figure 9 – Diagramme de séquence de la souscription d'un contrat

C. Service Sinistres (`claim-service` - MS3)

Le microservice claim-service gère la déclaration des sinistres (frais de soins) par les assurés, leur instruction par le gestionnaire et le calcul du montant remboursé. Il est développé avec Spring Boot (port 8083).
Il utilise une base PostgreSQL (claim_db). Un sinistre suit un cycle de statuts précis et porte des montants financiers : PostgreSQL garantit l'intégrité des données et les transactions lors de l'instruction, et conserve l'historique des sinistres de façon durable. Cette base n'est accessible que par ce microservice.

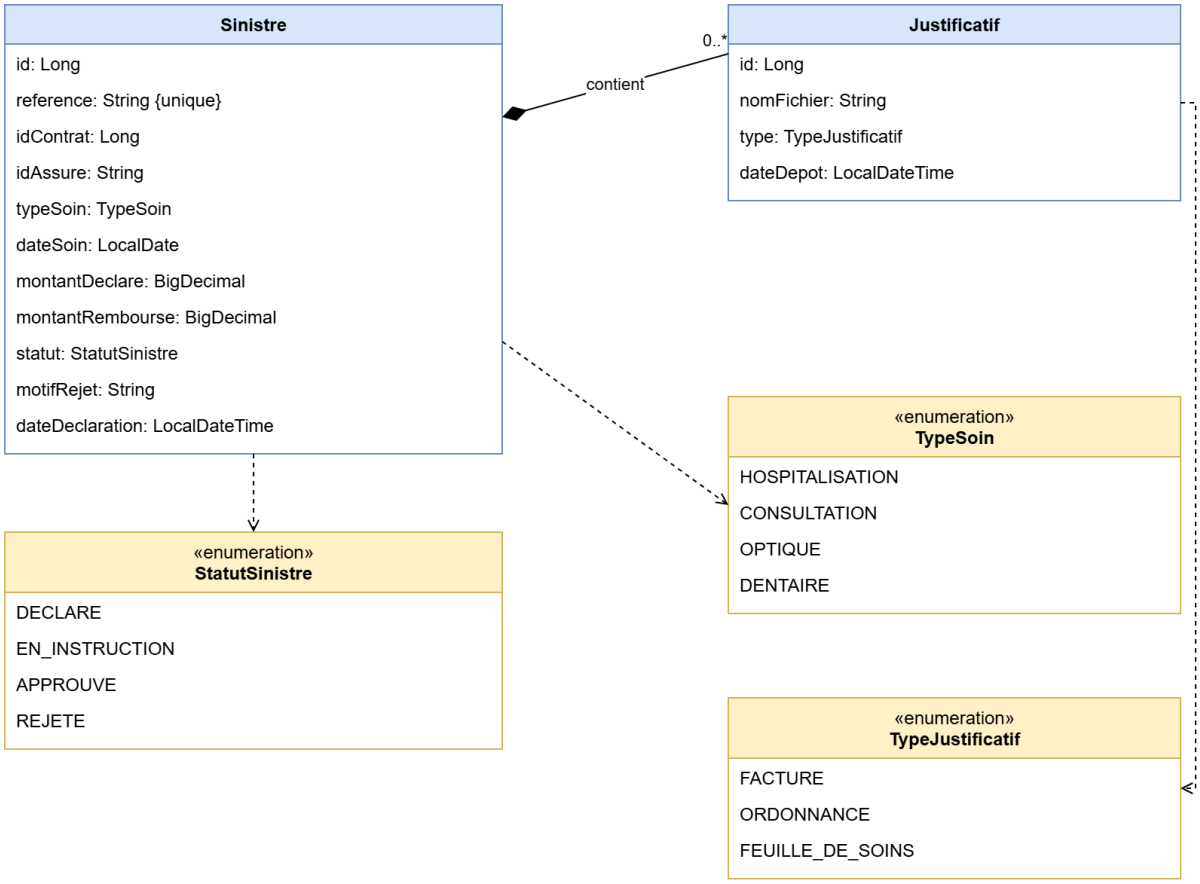


Figure 10 – Diagramme de classes du microservice claim-service

L'attribut idContrat référence un contrat de policy-service par son identifiant. L'identifiant idAssure est celui de l'utilisateur dans Keycloak : il est lu dans le jeton JWT, déjà vérifié par la Gateway. Aucun 2 microservice ne gère les comptes. Le montant remboursé est calculé à partir du taux de couverture lu dans policy-service, dans la limite du plafond annuel.

Contrat d'API :

Méthode	Endpoint	Description	Codes
POST	/api/claims	Déclarer un sinistre	201, 400, 404, 422, 503
GET	/api/claims/{id}	Détail d'un sinistre	200, 404
GET	/api/claims?idAssure=&idContrat=&statut=	Lister les sinistres	200
POST	/api/claims/{id}/justificatifs	Ajouter un justificatif	201, 404, 413
PATCH	/api/claims/{id}/instruction	Approuver ou rejeter	200, 404, 409

422 : le contrat n'est pas actif ; 409 : le sinistre a déjà été instruit ; 413 : fichier trop volumineux ; 503 : policy-service indisponible.

Événements :

Événement	Sens	Contenu	Effet
SinistreApprouve (sinistre.approuve)	Publié	idSinistre, idContrat, idAssure, typeSoin, montantRembourse	billing verse le remboursement ; predictive-scoring met à jour l'historique

Interactions :

Service	Mode	Objet	En cas d'indisponibilité
policy-service (appelé)	Synchrone (OpenFeign)	Vérifier que le contrat est actif et lire le taux de couverture	Erreur 503, déclaration refusée temporairement
billing-service	Asynchrone (RabbitMQ)	SinistreApprouve	Message conservé dans la file
predictive-scoring-service	Asynchrone (RabbitMQ)	SinistreApprouve	Message conservé dans la file

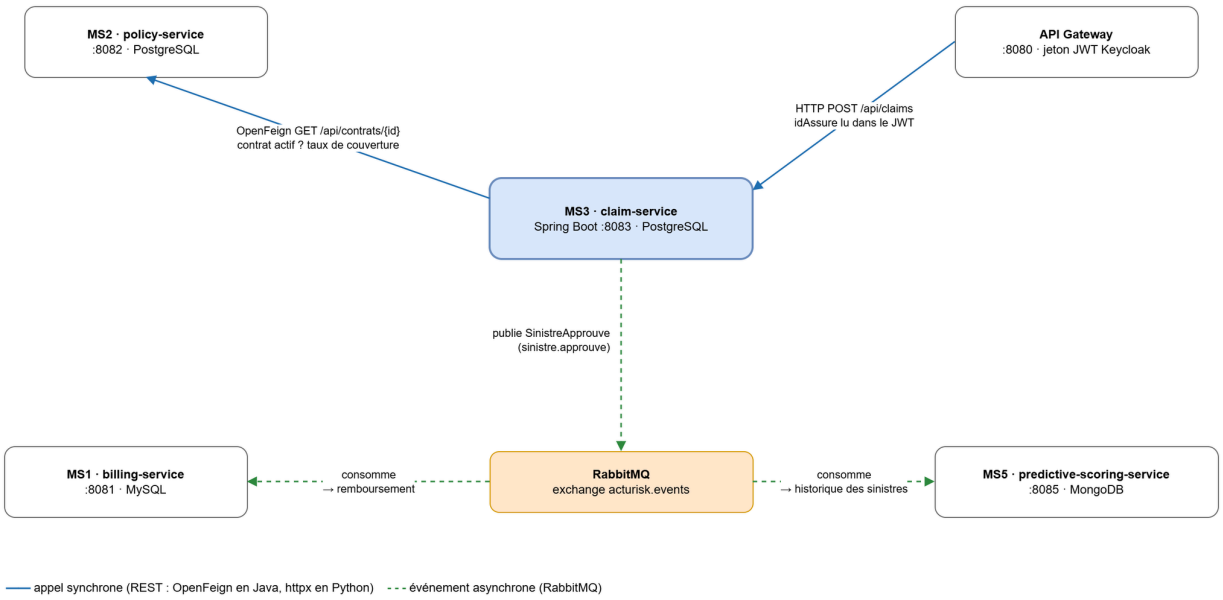


Figure 11 – Interactions du microservice claim-service

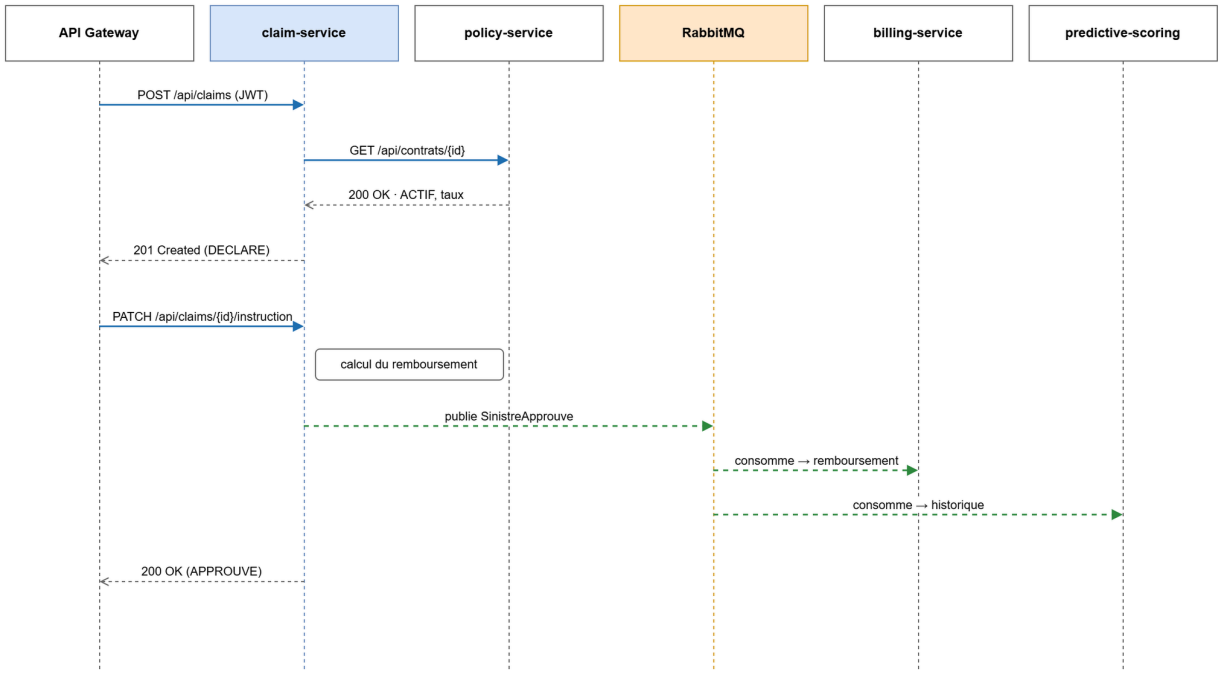


Figure 12 – Diagramme de séquence de la déclaration et de l'instruction d'un sinistre

D. Service Risque (`risk-service` - MS4)

Le microservice risk-service gère le profil de risque de chaque assuré (âge, taille, poids, tabac, niveau d'activité) et le paramétrage des facteurs de risque utilisés par l'actuaire. Il est développé avec Spring Boot (port 8084).
Il utilise une base PostgreSQL (risk_db). Les profils sont des données structurées avec une contrainte forte (un seul profil par assuré) ; PostgreSQL garantit cette intégrité et conserve les profils de façon durable. Cette base n'est accessible que par ce microservice.

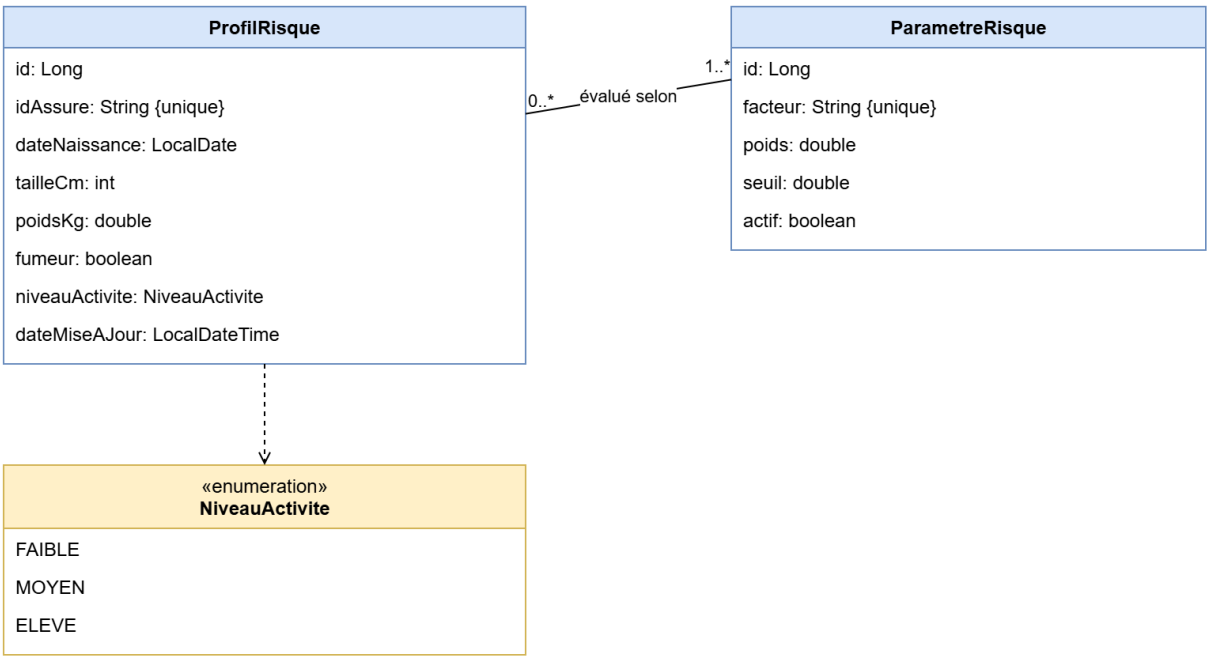


Figure 13 – Diagramme de classes du microservice risk-service

L'identifiant idAssure est celui de l'utilisateur dans Keycloak : il est lu dans le jeton JWT, déjà vérifié par la Gateway. Aucun microservice ne gère les comptes. Les paramètres de risque (par exemple « fumeur » ou « IMC élevé ») sont réglés par l'actuaire et servent au calcul du score dans predictive-scoring-service.

Contrat d'API :

Méthode	Endpoint	Description	Codes
POST	/api/risk-profiles	Créer le profil d'un assuré	201, 400, 409
GET	/api/risk-profiles/{idAssure}	Profil d'un assuré	200, 404
PUT	/api/risk-profiles/{idAssure}	Mettre à jour un profil	200, 400, 404
GET	/api/risk-parameters	Lister les paramètres	200
PUT	/api/risk-parameters/{id}	Modifier un paramètre	200, 400, 404

409 : un profil existe déjà pour cet assuré.

Événements :

Événement	Sens	Contenu	Effet
ProfilRisqueModifie (profil-risque.modifie)	Publié	idAssure, fumeur, imc, niveauActivite	predictive-scoring recalcule le score

Interactions :

Service	Mode	Objet	En cas d'indisponibilité
predictive-scoring-service (appelant)	Synchrone (HTTP, httpx)	Lire le profil de risque d'un assuré	—
predictive-scoring-service	Asynchrone (RabbitMQ)	ProfilRisqueModifie	Message conservé dans la file

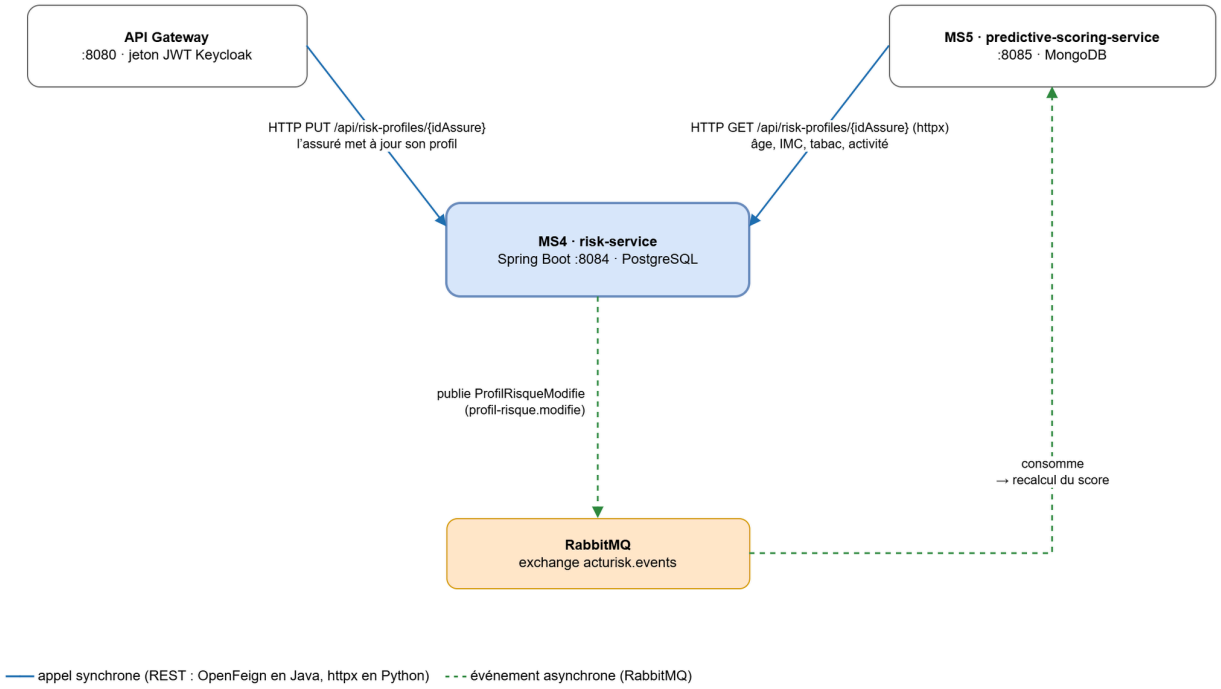


Figure 14 – Interactions du microservice risk-service

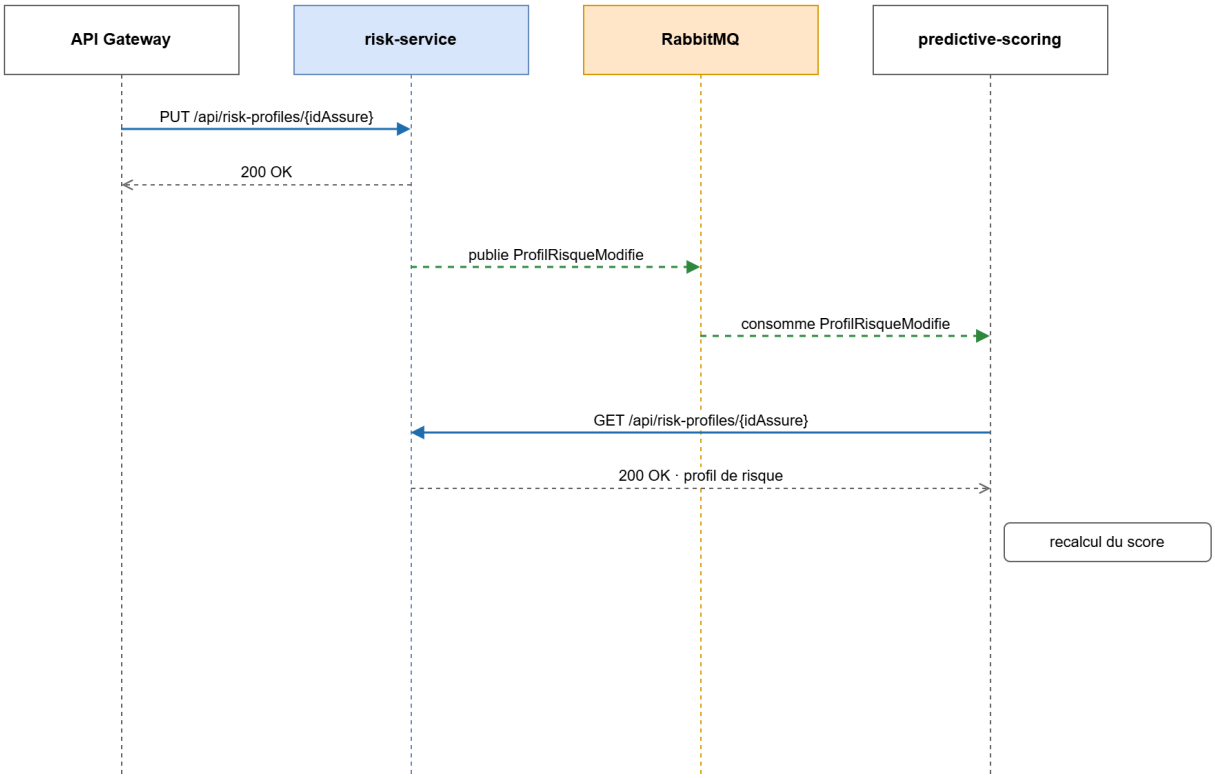


Figure 15 – Diagramme de séquence de la mise à jour d'un profil de risque

E. Service Commun & Scoring Prédictif (`predictive-scoring-service` - MS5)

Le microservice predictive-scoring-service calcule le score de risque de chaque assuré à partir de son profil de risque et de son historique de sinistres, puis propose une prime ajustée lors de la souscription d'un contrat. Il est développé en Python avec FastAPI (port 8085), ce qui permet d'utiliser une bibliothèque de machine learning pour le calcul du score.

Il utilise une base MongoDB (scoring_db) : un score contient une liste de facteurs variable et une version de modèle qui évolue, ce qui correspond mieux à un document qu'à une table au schéma fixe. Cette base n'est accessible que par ce microservice.

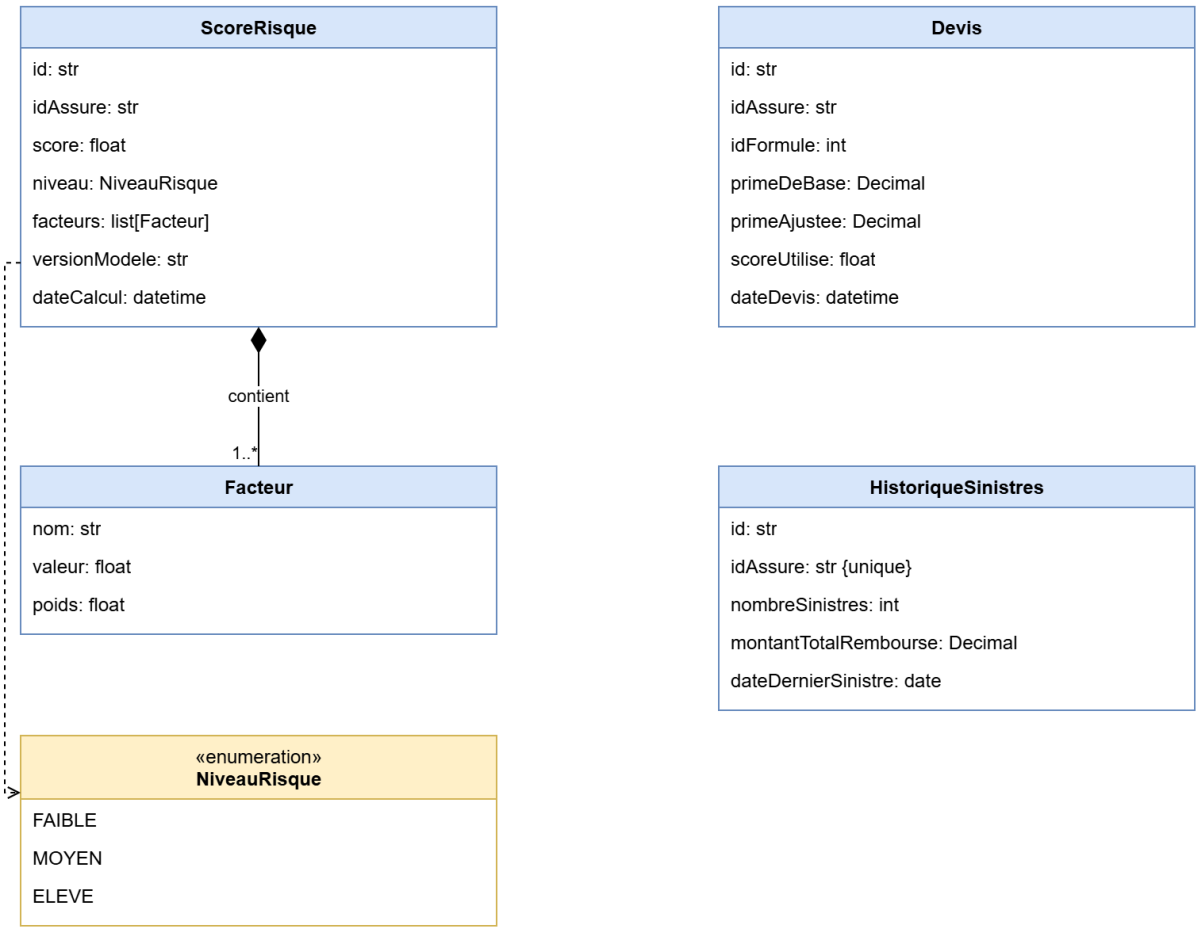


Figure 16 – Diagramme de classes du microservice predictive-scoring-service

Les documents sont indépendants et reliés à l'assuré par idAssure, l'identifiant Keycloak de l'utilisateur ; idFormule référence une formule de policy-service.

Contrat d'API :

Méthode	Endpoint	Description	Codes
POST	/api/scoring/devis	Calculer une prime ajustée	201, 400, 422
GET	/api/scoring/assures/{idAssure}	Dernier score d'un assuré	200, 404
GET	/api/scoring/assures/{idAssure}/historique	Historique des scores	200, 404
GET	/api/scoring/statistiques	Répartition par niveau de risque	200

422 : profil de risque incomplet.

Événements :

Événement	Sens	Contenu	Effet
ProfilRisqueModifie (profil-risque.modifie)	Reçu de risk-service	idAssure, fumeur, imc, niveauActivite	Recalcul du score
SinistreApprouve (sinistre.approuve)	Reçu de claim-service	idAssure, montantRembourse	Mise à jour de l'historique
ContratActive (contrat.active)	Reçu de policy-service	idContrat, idAssure, primeMensuelle	Mise à jour du portefeuille

Interactions :

Service	Mode	Objet	En cas d'indisponibilité
policy-service (appelant)	Synchrone (OpenFeign)	Demande de prime ajustée	policy-service applique la prime de base
risk-service (appelé)	Synchrone (HTTP, httpx)	Lire le profil de risque	Dernier score connu, sinon niveau MOYEN
risk-service	Asynchrone (RabbitMQ)	ProfilRisqueModifie	Message conservé dans la file
claim-service	Asynchrone (RabbitMQ)	SinistreApprouve	Message conservé dans la file
policy-service	Asynchrone (RabbitMQ)	ContratActive	Message conservé dans la file

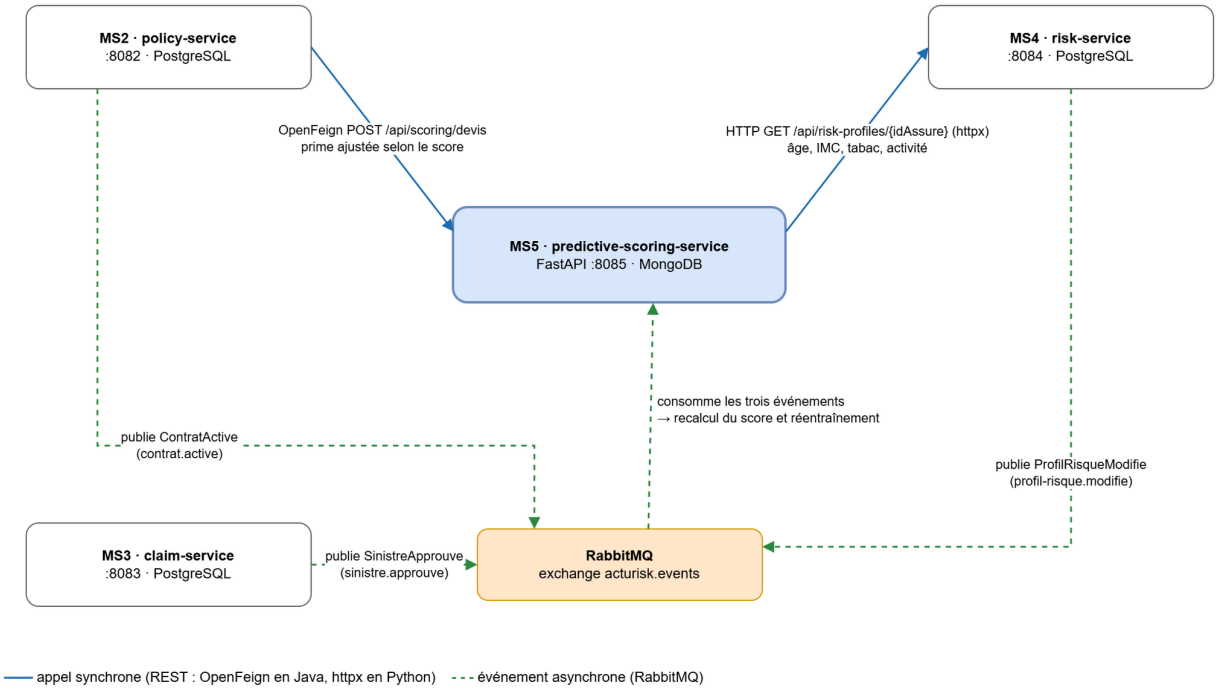


Figure 17 – Interactions du microservice predictive-scoring-service

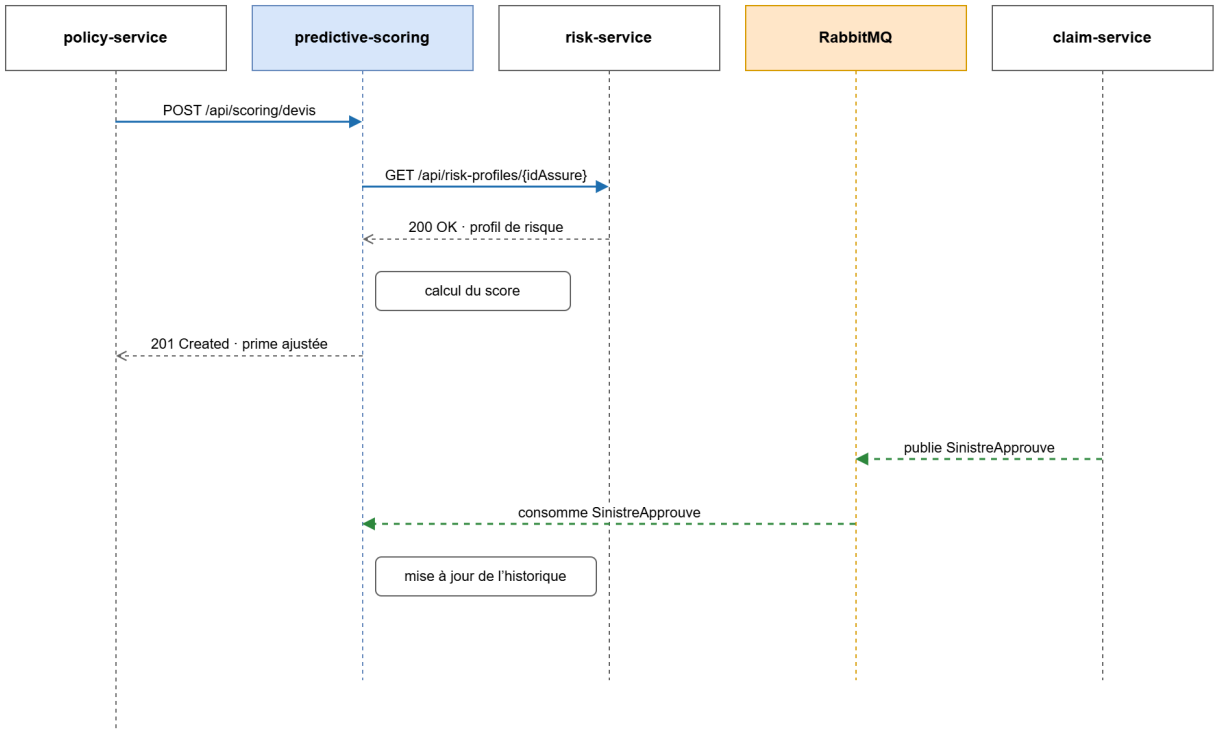


Figure 18 – Diagramme de séquence du calcul d'un devis

9. Diagramme de classe global :

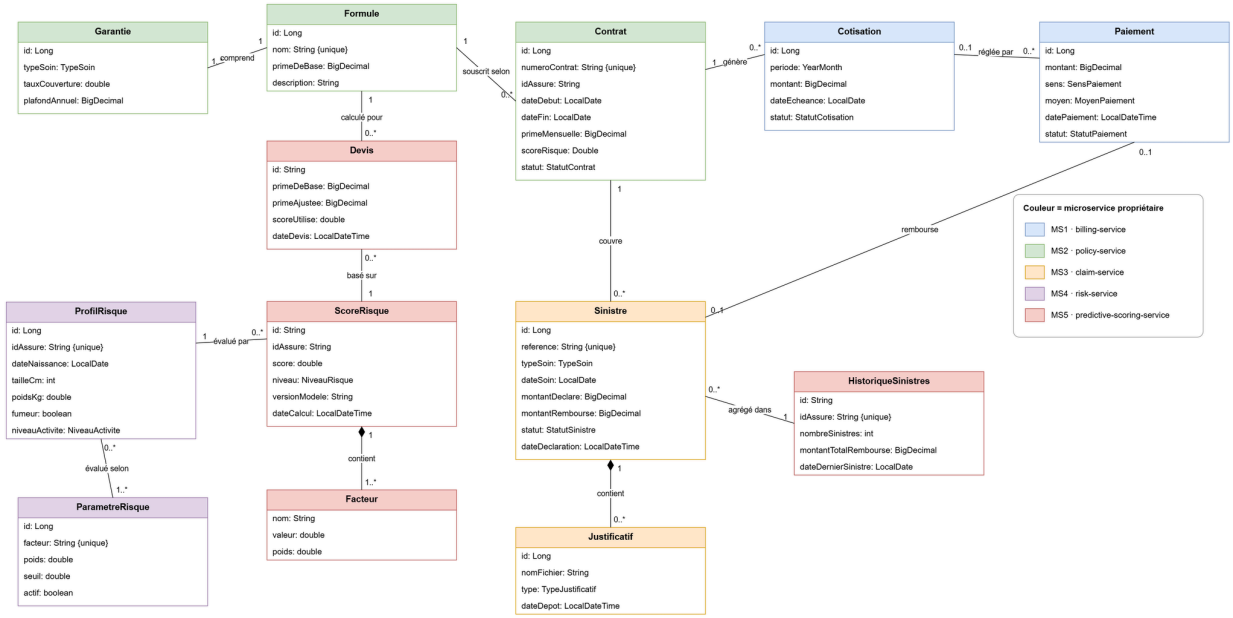


Figure 19 – Diagramme de classe global

Le Modèle de Données Global de la plateforme *ActuRisk* synthétise l'ensemble des entités métiers et techniques gérées par les différents microservices. Chaque domaine

est isolé autour de sa propre base de données (PostgreSQL, MySQL ou MongoDB) et identifié par un code couleur distinct. Les relations inter-services reposent sur des couplages par identifiants (tels que `idAssure` et `idContrat`), garantissant le découplage des architectures tout en assurant la traçabilité des données financières, contractuelles et actuarielles.